

Clipboard

Overview

XPF implements the WPF clipboard API (`System.Windows.Clipboard`) across all platforms. There are some differences from WPF's native Windows implementation that you should be aware of.

Basic usage

Standard WPF clipboard operations work in XPF. You can copy and retrieve text using `Clipboard.SetText` and `Clipboard.GetText`, or work with richer data through `DataObject` :

```
// Text
Clipboard.SetText("Hello, World!");
string text = Clipboard.GetText();

// Data object
var data = new DataObject();
data.SetData(DataFormats.Text, "Hello");
Clipboard.SetDataObject(data);
```



You can also check whether the clipboard contains a specific format before attempting to read it:

```
if (Clipboard.ContainsText())
{
    string text = Clipboard.GetText();
}
```



Bitmap support

Copying bitmaps to and from the clipboard is supported in XPF 1.6.0 and later:

```
// Copy bitmap to clipboard
Clipboard.SetImage(myBitmapSource);

// Retrieve bitmap from clipboard
BitmapSource image = Clipboard.GetImage();
```



On macOS, screenshots captured with system shortcuts (Cmd+Shift+Ctrl+3) may use pixel formats that differ from WPF conventions. XPF 1.6.0 and later automatically transcodes these to compatible formats.

Custom data formats

Custom clipboard data formats work within the same process. For cross-process clipboard operations with custom data, XPF serializes your data as strings. Make sure your custom data types are serializable:



```
var data = new DataObject();
data.SetData("MyCustomFormat", mySerializableObject);
Clipboard.SetDataObject(data);
```

To retrieve your custom data, use `Clipboard.GetDataObject` and call `GetData` with the same format string:

```
IDataObject clipboardData = Clipboard.GetDataObject();
if (clipboardData?.GetDataPresent("MyCustomFormat") == true)
{
    var result = clipboardData.GetData("MyCustomFormat");
}
```

STA threading (Windows)

On Windows, clipboard operations use COM and require the main thread to be marked as STA. If you encounter a `COMException` with the message `CoInitialize was not called`, make sure your entry point has the `[STAThread]` attribute:

```
[STAThread]
static void Main(string[] args)
{
    // Your application startup
}
```

Alternatively, you can use [custom initialization](#), which handles STA threading automatically.

Platform differences

The following table summarizes clipboard feature support across platforms:

Feature	Windows	macOS	Linux
Text	Supported	Supported	Supported
Bitmap	Supported (1.6.0+)	Supported (1.6.0+)	Supported (1.6.0+)
Custom formats (same process)	Supported	Supported	Supported
Custom formats (cross-process)	Supported (1.6.0+)	Supported (1.6.0+)	Supported (1.6.0+)
<code>Clipboard.Flush()</code>	Supported	No effect	Supported (X11)

`Clipboard.Flush()` persists clipboard data so it remains available after your application closes. On platforms where flushing is not supported, the method does nothing.

See also

- [Known differences](#)
- [Customizing initialization](#)
- [Troubleshooting](#)

 [Edit this page](#)

Last updated on Mar 7, 2026 by Mike James